

Manual operativo de datos

Clasificador de CVs con IA · Entrega Final · Arquitecto de Flujos IA · Rodrigo Benegas

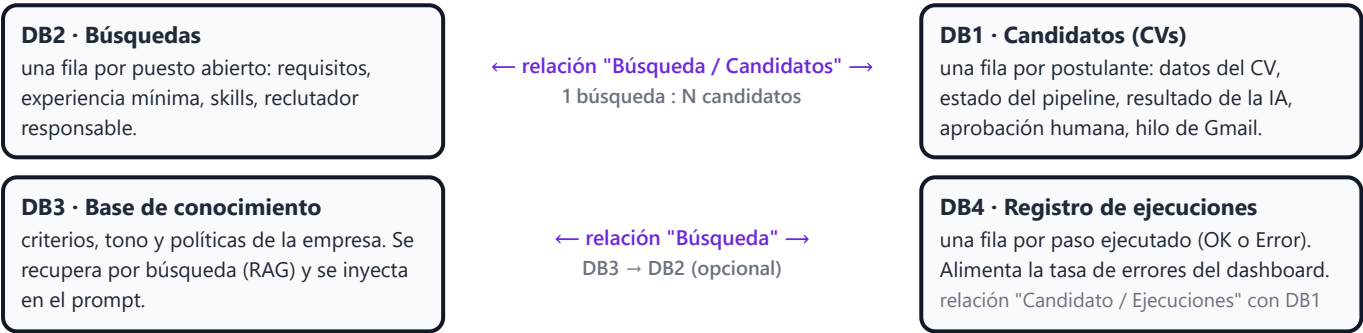
Memoria: Notion (4 bases relacionadas) **Orquestador:** Make · escenario "Entrega Final - Clasificador de CVs"

IA: openai/gpt-oss-120b vía Groq API **Canal:** Gmail

1. Para qué sirve este manual

Describe la memoria del sistema (las tablas de Notion, sus campos, sus relaciones y su ciclo de estados) y los esquemas JSON con los que la información viaja entre Notion, Make, la IA y Gmail. Es el documento que necesita una persona nueva para operar el flujo sin leer el blueprint.

2. Mapa de tablas y relaciones



Los datos de la búsqueda llegan a la ficha del candidato por **rollups** (Puesto, Requisitos, Skills clave, Exp. mínima, Reclutador). Así el flujo de Make lee todo desde un solo bundle (el de DB1) y no hay valores fijos en ningún módulo: cambiar el reclutador o los requisitos en DB2 cambia el comportamiento del flujo sin tocar Make.

3. Esquema de cada tabla

La columna **ID** es el identificador de la propiedad en la API de Notion; Make lo usa como clave en los módulos *Update a Database Item* y *Create a Database Item*. Los IDs fueron fijados al crear la estructura para que el blueprint sea estable.

DB1 · Candidatos (CVs) — tabla disparadora

Propiedad	Tipo	ID	La escribe	Uso
Nombre	title	title	Reclutador	Nombre de pila.
Apellido	text	fpLs	Reclutador	Apellido.
Cargo	text	LUHn	Reclutador	Cargo actual o último.
Experiencia (años)	number	SjjX	Reclutador	Años de experiencia. Es numérico para compararlo con la experiencia mínima (tipos correctos).
Correo electrónico	email	ZfS?	Reclutador	Destino de los emails de las rutas B y C. Nunca se envía a la IA.
Estado	status	m=>h	Reclutador y Make	Pendiente → En revisión → Contactado · Rechazado · Error. Es el campo que decide la ruta del Router.
Búsqueda	relation → DB2	Busq	Reclutador	A qué puesto se postula. Obligatoria para clasificar.
Resumen del CV	text	ResC	Reclutador	Texto del CV (o su resumen). Es lo que evalúa la IA. Obligatorio.
Clasificación IA	select	ClIA	Make	Apto · No apto · Revisar.
Puntaje IA	number	PtIA	Make	0 a 100.
Justificación IA	text	JsIA	Make	Por qué la IA decidió eso (≤ 400 caracteres).

Requisitos faltantes	text	Falt	Make	Lo que el CV no cubre.
Borrador de respuesta	text	Borr	Make (la IA) y el reclutador (lo edita)	Email propuesto al candidato. Se envía tal cual esté cuando el humano aprueba.
Aprobado	checkbox	Aprb	Reclutador	Compuerta HITL: nada sale al candidato hasta que esté marcada.
Notificado	checkbox	Ntfc	Make	Se marca cuando se envió el email de cierre (ruta C). Evita reenvíos.
Última ejecución	date	UltE	Make	Fecha/hora del último paso del flujo sobre esta ficha.
Error	text	Errr	Make	Texto del último error (API de IA, Gmail o datos incompletos). Se limpia cuando un paso sale bien.
Hilo Gmail	text	Hilo	Make	Thread ID del aviso enviado al reclutador. Permite responder en el mismo hilo.
Puesto · Requisitos · Skills clave · Reclutador	rollup (show original)	RPue RReq RSki RRec	Notion (automático)	Traen los datos de la búsqueda relacionada al bundle del trigger.
Exp. mínima	rollup (sum)	RExp	Notion	Experiencia mínima de la búsqueda, como número.
Ejecuciones	relation ← DB4	Ejec	Notion	Historial de pasos ejecutados sobre la ficha.

DB2 · Búsquedas

Propiedad	Tipo	ID	Uso
Puesto	title	title	Nombre del puesto. También es la clave con la que se recupera el contexto en DB3.
Requisitos	text	Requ	Excluyentes y deseables, en prosa.
Experiencia mínima	number	ExpM	Años. La IA resta 20 puntos si el candidato no la cumple.
Skills clave	text	Skil	Lista separada por comas.
Reclutador	email	Rec1	Quien recibe el aviso HITL. Variable dinámica: no hay direcciones fijas en Make.
Estado	select	EstB	Abierta · Cerrada.
Candidatos	relation ↔ DB1	Cand	Espejo de "Búsqueda".
Base de conocimiento	relation ↔ DB3	Base	Entradas de contexto asociadas.

DB3 · Base de conocimiento – Reclutamiento (RAG)

Propiedad	Tipo	ID	Uso
Tema	title	title	Nombre exacto del puesto al que aplica, o "General" para reglas de toda la empresa. Es el criterio de recuperación.
Tipo	select	Tipo	Criterios · Tono · Política.
Contenido	text	Cont	El texto que se inyecta en el prompt.
Búsqueda	relation → DB2	Busq	Trazabilidad (opcional).

DB4 · Registro de ejecuciones (log)

Propiedad	Tipo	ID	Uso
Ejecución	title	title	"dd/mm hh:mm · etapa · candidato".
Candidato	relation → DB1	Cand	Ficha sobre la que se ejecutó el paso.
Etapas	select	Etap	Clasificación IA · Notificación HITL · Envío al candidato · Rechazo · Validación de datos.
Resultado	select	Resu	OK · Error.

Manual operativo de datos - Clasificador de CVs con IA - Rodrigo Benegas - septiembre 2026

Error	checkbox	ErrC	Marcado en los errores. El dashboard calcula "% marcados" = tasa de errores .
Detalle	text	Deta	Resultado de la IA, thread id, o el mensaje de error de la API.
Modelo	text	Mode	Modelo usado (para la matriz de costos).
Tokens	number	Toke	Tokens totales de la llamada a la IA.
Fecha	created_time	Fech	Automática.

4. Ciclo de estados de una ficha

Estado actual	Condición adicional	Ruta de Make	Qué hace	Estado final
Pendiente	Resumen, Correo y Búsqueda cargados	A	RAG + IA + aviso al reclutador	En revisión
Pendiente	falta alguno de los tres	D (camino infeliz)	Marca qué falta, registra el error y avisa al reclutador	Error
En revisión	Aprobado ✓	B	Email al candidato con el borrador aprobado + respuesta en el hilo del reclutador	Contactado
Rechazado	Notificado vacío y Correo cargado	C	Email de cierre al candidato	Rechazado (Notificado ✓)
Error	—	ninguna	El humano corrige la ficha y vuelve a poner Pendiente	Pendiente
Contactado	—	ninguna	Fin del pipeline automático; el resto es humano	—

Anti-bucle. Cada ruta termina cambiando el Estado (o marcando Notificado). Como los filtros del Router exigen el estado *anterior*, la misma ficha no vuelve a entrar por la misma ruta aunque el trigger la detecte editada. Un ítem editado por Make consume como máximo 2 operaciones (trigger + router) sin efectos.

5. Esquemas JSON de transferencia

Los siete mensajes que circulan por el sistema, en el orden del flujo. Los valores son de ejemplo; los IDs largos están acortados.

5.1 Bundle del trigger — Notion → Make (*Watch Database Items*)

Make entrega cada ficha editada como un bundle. Los campos de texto llegan como arreglos de `rich_text` (por eso el flujo usa `join(map(...; "plain_text"); "")`), los selects como objeto con `name`, los números y checkboxes como valores simples y los rollups con la estructura de la API de Notion.

```
{
  "id": "3b4151a7-f4b6-81ab-...",
  "url": "https://www.notion.so/3b4151a7f4b681ab...",
  "last_edited_time": "2026-09-16T14:05:00.000Z",
  "properties_value": {
    "Nombre": [{ "plain_text": "Lucía" }],
    "Apellido": [{ "plain_text": "Fernández" }],
    "Cargo": [{ "plain_text": "Backend Developer" }],
    "Experiencia (años)": 5,
    "Correo electrónico": "lucia@ejemplo.com",
    "Estado": { "name": "Pendiente", "color": "yellow" },
    "Búsqueda": [{ "id": "...id de la búsqueda..." }],
    "Resumen del CV": [{ "plain_text": "5 años en Node.js, APIs REST, PostgreSQL..." }],
    "Puesto": { "type": "array", "array": [{ "type": "title", "title": [{ "plain_text": "Desarrollador Backend Node.js" }]} ] },
    "Exp. mínima": { "type": "number", "number": 3 },
    "Reclutador": { "type": "array", "array": [{ "type": "email", "email": "reclutador@empresa.com" }]} ],
    "Aprobado": false,
    "Notificado": false
  }
}
```

5.2 Filtro del Router — decide la ruta (tipos correctos)

Los filtros comparan **texto con texto** (`text:equal`), **booleanos con booleanos** (`boolean:equal`) y usan `exist` / `notexist` para detectar datos faltantes. Ejemplo de la ruta A:

```
{
  "name": "A · Pendiente + datos completos",
  "conditions": [[
    { "a": "{{1.properties_value.Estado.name}}", "b": "Pendiente", "o": "text:equal" },
    { "a": "{{join(map(1.properties_value.`Resumen del CV`; \"plain_text\"); \"\")}}", "o": "exist" },
    { "a": "{{1.properties_value.`Correo electrónico`}}", "o": "exist" },
    { "a": "{{1.properties_value.Búsqueda[id]}}", "o": "exist" }
  ]]
}
```

5.3 Recuperación de contexto — Make → Notion (*Search Objects sobre DB3*)

Se piden las entradas cuyo Tema es el puesto de la búsqueda o "General" (máximo 10). El Text Aggregator las une en un solo texto que va al prompt.

```
{
  "filter": { "or": [
    { "property": "Tema", "title": { "equals": "Desarrollador Backend Node.js" } },
    { "property": "Tema", "title": { "equals": "General" } } ] ],
  "page_size": 10
}
```

5.4 Llamada a la IA — Make → Groq API (*Create a Chat Completion*)

Prompt de sistema con el contrato de salida; prompt de usuario con la búsqueda, el candidato y el contexto RAG. Salida forzada a JSON y **máximo 700 tokens**.

```
{
  "model": "openai/gpt-oss-120b",
  "messages": [
    { "role": "system", "content": "Sos un analista senior de selección de personal... Respondé ÚNICAMENTE con un objeto JSON válido..." },
    { "role": "user", "content": "## Búsqueda\nPuesto: Desarrollador Backend Node.js\nRequisitos: ...\n## Candidato\nNombre: Lucía Fernández\n...\n## Base de conocimiento de la empresa\n### Criterios · General\n..." }
  ],
  "max_tokens": 700,
  "temperature": 0.2,
  "response_format": { "type": "json_object" }
}
```

5.5 Contrato de salida de la IA — Groq → Make (estructura de datos "Salida clasificador IA")

El módulo `Parse JSON` valida la respuesta contra esta estructura. Si la IA devuelve otra cosa, el error handler registra el problema y deja la ficha en Error.

Campo	Tipo	Obligatorio	Regla
<code>clasificacion</code>	text	sí	"Apto" (≥ 75) · "Revisar" (50-74) · "No apto" (< 50)
<code>puntaje</code>	number	sí	entero 0-100; -20 si no cumple la experiencia mínima
<code>justificacion</code>	text	sí	≤ 400 caracteres, cita hechos del CV
<code>borrador_respuesta</code>	text	sí	email al candidato, tono cordial, sin promesas
<code>faltantes</code>	text	no	requisitos no cumplidos o "ninguno"

```
{
  "clasificacion": "Apto",
  "puntaje": 82,
  "justificacion": "5 años con Node.js y PostgreSQL, APIs REST en producción; no menciona AWS.",
  "faltantes": "AWS",
  "borrador_respuesta": "Hola Lucía, gracias por postularte... ¿Tenés disponibilidad esta semana para una entrevista técnica de 45 minutos? ... Equipo de Selección"
}
```

5.6 Aviso HITL — Make → Gmail (*Send an Email*) y el Thread ID

El aviso al reclutador lleva un `Message-ID` propio, derivado del ID de la ficha. La salida del módulo devuelve el `threadId`, que se guarda en la ficha (campo Hilo Gmail). Cuando el candidato es contactado, el flujo responde al reclutador **en ese mismo hilo** con `In-Reply-To` y `References`, sin crear correos huérfanos.

```
// entrada
{ "to": ["reclutador@empresa.com"],
  "subject": "[Revisión IA] Lucía Fernández · Desarrollador Backend Node.js · Apto (82/100)",
  "bodyType": "rawHtml",
  "emailHeaders": [{ "key": "Message-ID", "value": "<cand-3b4151a7-f4b6-81ab-...@clasificador-cvs.make>" }],
  "content": "<div>Revisión de candidato... <a href='{{1.url}}'>Abrir la ficha en Notion</a>...</div>" }
// salida
{ "id": "18f3a2c9e4b1d7a0", "threadId": "18f3a2c9e4b1d7a0", "labelIds": ["SENT"] }
// respuesta en el hilo (ruta B, módulo 13)
{ "subject": "Re: [Revisión IA] Lucía Fernández · Desarrollador Backend Node.js · Apto (82/100)",
  "emailHeaders": [{ "key": "In-Reply-To", "value": "<cand-...@clasificador-cvs.make>" },
    { "key": "References", "value": "<cand-...@clasificador-cvs.make>" } ] }
```

5.7 Escritura en Notion — Make → Notion (*Update DB1* y *Create DB4*)

Make traduce las claves (IDs de propiedad) al formato de la API de Notion. Así queda la ficha después de la ruta A y el registro que se crea en el log:

```
// PATCH /v1/pages/{id candidato}
Manual operativo de datos · Clasificador de CVs con IA · Rodrigo Benegas · septiembre 2026
{ "properties": {
  "Clasificación IA": { "select": { "name": "Apto" } },
  "Puntaje IA": { "number": 82 },
  "Justificación IA": { "rich_text": [{ "text": { "content": "5 años con Node.js..." } }] },
  "Requisitos faltantes": { "rich_text": [{ "text": { "content": "AWS" } }] },
  "Borrador de respuesta": { "rich_text": [{ "text": { "content": "Hola Lucía..." } }] },
  "Estado": { "status": { "name": "En revisión" } },
  "Hilo Gmail": { "rich_text": [{ "text": { "content": "18f3a2c9e4b1d7a0" } }] },
  "Última ejecución": { "date": { "start": "2026-09-16T14:06:12.000Z" } },
  "Error": { "rich_text": [] } }

// POST /v1/pages (registro en DB4)
{ "parent": { "database_id": "...DB4..." },
  "properties": {
    "Ejecución": { "title": [{ "text": { "content": "16/09 11:06 · Clasificación IA · Lucía Fernández" } }] },
    "Candidato": { "relation": [{ "id": "3b4151a7-f4b6-81ab-..." } ] },
    "Etapa": { "select": { "name": "Clasificación IA" } },
    "Resultado": { "select": { "name": "OK" } },
    "Error": { "checkbox": false },
    "Detalle": { "rich_text": [{ "text": { "content": "Apto (82/100) · aviso al reclutador enviado, hilo 18f3a2c9e4b1d7a0" } }] },
    "Modelo": { "rich_text": [{ "text": { "content": "openai/gpt-oss-120b" } }] },
    "Tokens": { "number": 1934 } } }
```

En un error, el handler crea el mismo registro con "Resultado": "Error", "Error": true y el mensaje de la API en Detalle, y deja la ficha con "Estado": "Error" y el texto en el campo Error.

6. Operación diaria

Abrir una búsqueda

En DB2 crear la fila con Puesto, Requisitos, Experiencia mínima, Skills clave y el email del Reclutador. Si el puesto tiene criterios propios, agregar en DB3 entradas con Tema = nombre exacto del puesto.

Cargar un candidato

En DB1: Nombre, Apellido, Cargo, Experiencia (número), Correo, Búsqueda (relación) y Resumen del CV. Estado = Pendiente. En menos de 15 minutos la ficha pasa a "En revisión" y llega el email de revisión.

Aprobar o rechazar

Leer el email, abrir la ficha, editar el Borrador de respuesta si hace falta y marcar **Aprobado** ✓ (envía la invitación) o poner **Estado = Rechazado** (envía el cierre). El sistema nunca escribe al candidato sin ese clic.

Si una ficha queda en Error

Leer el campo Error (qué falta o qué API falló). Corregir y volver a poner Estado = Pendiente. Los errores de API se reintentan solos 3 veces cada 15 minutos antes de necesitar intervención.

Qué mirar en el dashboard

- **Tasa de errores** (DB4, % de filas con Error ✓): objetivo < 10 %. Si sube, revisar el detalle de las últimas filas: suele ser cuota de Groq/Gmail o fichas incompletas.
- **Fichas en "En revisión"**: son las que esperan al humano. Si se acumulan, el cuello de botella es la revisión, no el flujo.
- **Tokens por ejecución**: si crece, revisar el largo de los resúmenes de CV o de las entradas de DB3.

Convenciones

- Todos los módulos de Make tienen nombre propio (número + acción) y toman sus valores del bundle: no hay emails, nombres de puesto ni IDs de páginas escritos a mano.
- Los IDs de propiedad son fijos: renombrar una propiedad en Notion no rompe el flujo; borrarla y crearla de nuevo sí.
- El blueprint exportado (blueprint/clasificador-cvs.blueprint.json) no contiene credenciales, solo referencias a las conexiones de Make.